Exercício 10.3: Depuração: aplicação matando conexões no rolling update

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, namespace `store-dev`, kube-proxy em modo iptables, Calico.
- Imagens em `app/`, todas com o código da `store-api`, em pares `-a` / `-b` para o `set image` sempre trocar a tag:
- `shell-a/b`: `ENTRYPOINT` em shell form (`Dockerfile-shell`)
- `exec-a/b`: `ENTRYPOINT` em exec form (`Dockerfile-exec`)
- `life-a/b`: exec form mais o `lifespan` no FastAPI (`Dockerfile-lifespan`, `main-lifespan.py`)
- Medição automatizada por `09-roda-etapas.sh`. Para cada etapa, o script sobe a tag `-a` e espera 15 s para os Pods antigos terminarem. Depois dispara **600 requisições sequenciais** (`07-carga.sh`, `wget -T 2` a cada 0,1 s, cerca de 64 s no total) e, 5 s depois do início da carga, faz o `set image` para a tag `-b`. A carga cobre o rollout inteiro.

YAML quebrado e YAML final

```yaml
# quebrado: shell form, sem probe, sem preStop, grace 5, maxUnavailable 1
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: store-api
  strategy:
```

```
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: store-api
    spec:
      terminationGracePeriodSeconds: 5
      containers:
        - name: store-api
          image: store-api:shell-a
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  selector:
    app: store-api
  ports:
    - port: 80
      targetPort: http
```

```
# (e) + lifespan: /ready 503 no shutdown (versao final)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: store-api
  strategy:
```

```
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: store-api
    spec:
      terminationGracePeriodSeconds: 45
      containers:
        - name: store-api
          image: store-api:life-a
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 2
            failureThreshold: 1
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "10"]
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  selector:
    app: store-api
  ports:
    - port: 80
      targetPort: http
```

As etapas intermediárias (`02` a `05`) adicionam uma correção de cada vez sobre a anterior.

Gerador de carga

```
#!/bin/sh
# gerador de carga: N requisicoes sequenciais ao Service, timeout 2s, ~10/s
# uso (dentro do Pod carga): sh /carga.sh 600
N=${1:-600}
i=0
while [ $i -lt $N ]; do
  if wget -q -O /dev/null -T 2 http://store-api/versao; then echo OK; else echo FALHA; fi
  i=$((i + 1))
  sleep 0.1
done | sort | uniq -c
```

Resultado por etapa

Etapa	Correção acumulada	Falhas / 600	Falhas / 100
quebrado	shell form, sem probe, sem preStop, grace 5, `maxUnavailable: 1`	**3, 7, 7, 9** (4 rodadas)	0,5 a 1,5
(a)	+ `readinessProbe` em `/ready`	0	0
(b)	+ `preStop: sleep 10` e grace 45	0	0
(c)	+ `ENTRYPOINT` exec form	0	0
(d)	+ `maxUnavailable: 0`, `maxSurge: 1`	0	0
(e) final	+ `lifespan` com `/ready` 503 no shutdown	**0, 0** (2 rodadas)	0
extra	(a) **com exec form** e sem preStop	**3, 4, 1** (3 rodadas)	0,2 a 0,7

A versão final passou por **2 rollouts completos com 600 de 600** requisições OK cada (o critério pede 500 consecutivas).

Achado: o shell form mascarava a corrida na etapa (a)

A primeira leitura da tabela sugere que só a `readinessProbe` resolve tudo. Isso é falso, e a linha ”extra” prova. A diferença está em quem é o PID 1:

```
## shell form (store-api:shell-a)
1 /bin/sh -c sh -c "uvicorn main:app --host 0.0.0.0 --port 8000"
7 sh -c uvicorn main:app --host 0.0.0.0 --port 8000
8 /usr/local/bin/python3.12 /usr/local/bin/uvicorn main:app --host 0.0.0.0 --port 8000

## exec form (store-api:exec-a)
1 /usr/local/bin/python3.12 /usr/local/bin/uvicorn main:app --host 0.0.0.0 --port 8000

$ docker stop -t 15 t-shell   →  15.12 s, exit=137 (SIGKILL)
$ docker stop -t 15 t-exec    →   0.27 s, exit=0
```

No shell form, o PID 1 é o `sh`. Como PID 1, ele não tem handler padrão para SIGTERM, ignora o sinal e não repassa para o uvicorn. O uvicorn **continua servindo** até o SIGKILL no fim do grace period. Na etapa (a) isso dava ao Pod 5 s de sobrevida após o SIGTERM, tempo suficiente para os Endpoints e o iptables de todos os nós serem atualizados. Por acidente, o defeito escondia a corrida. Ao trocar só a imagem para exec form, mantendo a probe e sem preStop, o uvicorn passa a sair em milissegundos e as falhas voltam (3, 4 e 1 em 600). Ou seja, (c) sem (b) **piora** o rollout. É por isso que a ordem das correções importa e que o preStop é a peça central.

O preStop no Pod em `Terminating`

Saída da etapa (b), 4 s depois do `set image`:

```
$ kubectl -n store-dev describe pod store-api-65ccd9b7f9-c7dgf
Status:                    Terminating (lasts <invalid>)
Termination Grace Period:  45s
Containers:
  store-api:
    Image:          store-api:shell-a
    State:          Running
    Ready:          True
    Readiness:      http-get http://:http/ready delay=0s timeout=1s period=2s #success=1 #failure=1
Events:
  Normal   Killing    2s    kubelet  spec.containers{store-api}: Stopping container store-api
```

O kubectl 1.36 **não imprime os hooks de lifecycle** no `describe`, então o hook foi conferido no próprio objeto do Pod em `Terminating` (versão final):

```
$ kubectl -n store-dev get pod store-api-5bc7bccf9f-47pfs \
    -o jsonpath='{.spec.containers[0].lifecycle}{"\n"}{.metadata.deletionTimestamp}'
{"preStop":{"exec":{"command":["sleep","10"]}}}
2026-10-24T04:30:35Z
```

O Pod está `Terminating` e ainda `Running` / `Ready: True`, que é exatamente a janela do `sleep 10`. O `deletionTimestamp` é o prazo final (início da terminação + 45 s de grace).

Achado: a etapa (e) não tem efeito

```
$ docker run -d --name tl store-api:life-a; docker stop tl; docker logs tl
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [1]
lifespan shutdown: PRONTO=False
```

O `lifespan` do FastAPI roda o código pós-`yield` **depois** que o uvicorn já fechou o socket (`Shutting down`). Quando `PRONTO` vira `False`, ninguém mais consegue chamar o `/ready`, então o 503 nunca é servido. Além disso, com o preStop, o SIGTERM só chega 10 s depois do início da terminação, e nesse momento o Pod já saiu dos Endpoints. Para o `/ready` responder 503 de forma útil, a aplicação teria de tratar o SIGTERM **antes** de o uvicorn parar (um handler de sinal próprio) ou ter um endpoint de "drain" chamado pelo próprio preStop. Os 0 de 600 da etapa (e) vêm das etapas anteriores, não do lifespan.

Por que os Endpoints e o SIGTERM correm em paralelo

Quando um Pod é apagado, o API server só grava o `deletionTimestamp`, e **dois controladores independentes** reagem ao mesmo evento sem se coordenar. De um lado, o kubelet do nó começa a terminação: roda o preStop, se houver, e então manda SIGTERM ao container. Do outro, o EndpointSlice controller tira o Pod do slice, e só depois disso o kube-proxy de **cada nó** reescreve suas regras de iptables. Esse segundo caminho tem várias etapas assíncronas (watch, controller, API, watch de novo, kube-proxy, iptables em N nós) e quase sempre é mais lento que o primeiro, que é local. Sem preStop, o processo recebe SIGTERM e para de aceitar conexões enquanto os outros nós ainda roteiam tráfego para o IP dele, e daí vêm os `Connection refused` e os timeouts medidos. A `readinessProbe` não resolve, porque ela age na **entrada** de um Pod novo, não na saída de um antigo. O `preStop: sleep 10` não "conserta" a corrida, ele **dá vantagem** para o lado lento: o processo segue atendendo por 10 s enquanto a remoção se propaga. O grace de 45 s precisa caber o preStop mais o shutdown da aplicação.

Critérios de aceite

Critério	Resultado
Versão quebrada com falhas mensuráveis (> 0) no rollout	ok (3, 7, 7 e 9 em 4 rodadas)
Versão final com zero falhas em pelo menos 500 requisições consecutivas num rollout completo	ok (600/600 em 2 rollouts)
Explicação da corrida entre Endpoints/iptables e SIGTERM	ok, com a prova extra de que o shell form a mascarava